Token-Efficiency Measurement — rock-solid anti-bluff proof methodology

Revision: 1 **Last modified:** 2026-06-09T08:30:00Z **Status:** design **Authority:** constitution submodule, universal (§11.4.17). Composes with §11.4.5 / §11.4.6 (no-guessing — measured, never asserted) / §11.4.50 (deterministic consistency) / §11.4.69 (captured-evidence taxonomy) / §1.1 (paired mutation). The headline 60–70% number is NEVER claimed without this harness producing the captured proof.

---

0. The anti-bluff bar

Per §11.4.6 and §11.4.123, the token-efficiency claim is itself subject to the anti-bluff covenant: a "60–70% reduction" statement is a **bluff** unless it is backed by a captured `tokens-per-development-cycle` measurement on a fixed workload, BEFORE vs AFTER, with the input/output/cached split, produced by a repeatable harness, and reproduced N times (§11.4.50) to identical verdict. The pass criterion is mechanical: **AFTER total cost ≤ 40% of BEFORE total cost** (target) OR the measured best-safe reduction with a cited reason it cannot reach 40% (cold-cache floor per RESEARCH §5).

1. The ground-truth token source

Only the Anthropic `usage` object and `count_tokens` endpoint are authoritative. `tiktoken` is forbidden (undercounts Claude 15–20% + — `shared/token-counting.md`). The per-request `usage` object gives the exact split:

Field	Meaning	Price multiplier
`input_tokens`	uncached input	1.0×
`cache_creation_input_tokens`	written to cache this request	1.25× (5m) / 2× (1h)
`cache_read_input_tokens`		0.1×

Field	Meaning	Price multiplier
	served from cache	
`output_tokens`	generated	5× input price (Opus)

Cost formula (per request), using the model's input price `Pin` and output price `Pout` from `shared/models.md`:

```
cost = Pin × (input_tokens
             + 0.1 × cache_read_input_tokens
             + 1.25 × cache_creation_input_tokens_5m
             + 2.0  × cache_creation_input_tokens_1h)
     + Pout × output_tokens
```

(Opus 4.8: Pin = $5/1M, Pout = $25/1M. Sonnet 4.6: $3/$15. Haiku 4.5: $1/$5.) For multi-model cycles (subagent tiering), sum per-model with that model's prices — this is how M2's savings show up.

2. The fixed measurement workload (the "development cycle")

A **frozen, scripted, deterministic** workload so BEFORE and AFTER are comparable (§11.4.50). Recommended cycle = a representative slice of real project work, fixed and version-pinned: 1. Session start (governance loaded). 2. N1 structural-navigation turns (e.g. "where is X / what calls Y") — exercises M4. 3. N2 file-read + small-edit turns — exercises M3/M6. 4. N3 mechanical-subagent dispatches (search/grep/status/doc-export) — exercises M2/M5. 5. N4 reasoning/review turns (strong model) — exercises that the strong path is untouched. 6. Session end.

The workload is captured as a script (fixed prompts, fixed file targets, fixed expected actions) so it replays identically. It is committed under `constitution/docs/research/token_efficiency/workload/` (or the consuming project's equivalent) so any agent can re-run it.

3. The harness

`token_accounting.sh` (or a small Go/Python program — universal, project supplies model IDs): 1. Runs the fixed workload against the agent. 2. Captures every request's `usage` object (via the SDK `usage` field, or Claude Code's `--output-format json` / cost-tracking surface — `https://code.claude.com/docs/en/agent-sdk/cost-tracking`; note `total_cost_usd` is a client-side estimate, so the harness

recomputes cost from the raw token fields with the price table, never trusts the estimate). 3. Aggregates per cycle: `Σ input_tokens`, `Σ cache_read_input_tokens`, `Σ cache_creation_input_tokens`, `Σ output_tokens`, and the computed `cost` (§1 formula), split by model. 4. Emits `cycle_report.json` (captured-evidence artefact per §11.4.69 — feature class implied: `token_efficiency`).

4. Baseline capture (BEFORE)

Run the harness on the **current** configuration (governance inlined, no tiering, no index) N=3 times (§11.4.50). Record `BEFORE = mean(cost)` and assert the 3 runs' token splits are identical (deterministic workload ⇒ identical splits; any divergence is a workload-nondeterminism bug to fix first). Commit `cycle_report.before.json`.

5. After capture + pass criterion

Apply the M1–M7 measures, re-run the harness N=3, record `AFTER = mean(cost)`, assert the 3 AFTER runs identical (§11.4.50). Then:

```
reduction = 1 - (AFTER / BEFORE)
PASS  if  AFTER ≤ 0.40 × BEFORE        (≥ 60% reduction — target
met)
WARN  if  0.40×BEFORE < AFTER ≤ 0.55×BEFORE  AND  cache-cold
reason cited (RESEARCH §5 floor)
FAIL  otherwise (or if any quality/safety regression — see §6)
```

Commit `cycle_report.after.json` + a one-line `verdict.txt` citing the measured `reduction`, the `cache_read_input_tokens` warm-proof, and the per-model split that shows the tiering saving. **The number in any report/commit/anchor MUST equal the measured `reduction`, never the design estimate** (§11.4.6).

6. The non-regression (quality/safety) companion — mandatory

Cost reduction with quality regression is a §11.4 FAIL, not a win. The AFTER run MUST ALSO show **zero regression** against BEFORE on: - **Pre-build full sweep** (`pre_build_verification.sh` equivalent) — same PASS/FAIL set (§11.4.40). - **Meta-test mutation sweep** — every gate still FAILs its paired mutation (§1.1) — proves no gate became a bluff because its rule text moved to an index (M3 safety). - **Propagation gates** — every `CM-COVENANT-114-N-PROPAGATION` still passes (literal anchors intact after M3). - **A reasoning-path probe** — one fixed reasoning/review

turn produces an equivalent-quality verdict on the strong model (proves M2/M5 did not tier-down a decision path). - **Cache-warm proof** — `cache_read_input_tokens > 0` on the governance-bearing turns (proves M1 engaged, not silently invalidated).

If any of these regress, the cost win is rejected (FAIL) regardless of the cost number.

7. Repeatability / determinism (§11.4.50)

The whole BEFORE/AFTER measurement is itself wrapped in the project's `ab_run_n_times`-style N-iteration check: same workload + same model + same config ⇒ identical token splits and identical verdict across N runs. A divergent run is auto-FAIL (the workload must be deterministic before any cost claim is valid). Default N=3; cycle-validation N=10.

8. Paired §1.1 mutation (proves the harness is not a bluff)

The recommended `CM-TOKEN-EFFICIENCY` gate's paired mutation: **inject a per-turn timestamp/UUID into the governance prefix ahead of the cache breakpoint** (a real silent invalidator). The harness MUST then observe `cache_read_input_tokens` collapse toward 0 and the measured `reduction` fall below the pass bar → the gate FAILs. Restoring the stable prefix → gate PASSes. This proves the harness actually measures caching, not a hardcoded green.

9. What "rock-solid proof" means here

The claim ships with: `cycle_report.before.json`, `cycle_report.after.json`, `verdict.txt` (measured reduction + warm-cache evidence + per-model split), the N-iteration determinism log, the non-regression sweep result, and the paired-mutation demonstration. No estimate, no `tiktoken`, no trusting the client-side `total_cost_usd` — only the raw `usage` token fields, recomputed with the published prices, on a frozen workload, reproduced N times. That is the §11.4.6 / §11.4.123 bar.

Sources

- Token counting — Claude API Docs
- Track cost and usage — Claude Agent SDK

- `claude-api` skill `shared/prompt-caching.md` (usage-field semantics, 0.1×/1.25×/2× multipliers, silent-invalidator detection via `cache_read_input_tokens`), `shared/token-counting.md` (tiktoken forbidden; `count_tokens` authoritative), `shared/models.md` (per-model input/output prices).